Rocket Mission Simulator - Final Product Backlog

#	Name	How to demo	Notes
1	**JavaFX application and GUI layout**	Start the application. Verify that the central simulation area, telemetry sidebar, mission controls and mission log all appear together.	Created the Maven/JavaFX project and BorderPane layout, then separated the main interface into dedicated JavaFX panes.
2	**Rocket graphics and launch environment**	Open the simulator. Check that the rocket, ground and sky are visible; launch and observe the scenery move as the rocket ascends.	Added and positioned the Starship image, basic flight background and altitude-driven scenery movement. Rocket placement adjusts to the view.
3	**Mission states and launch controls**	Press Launch. The status changes from pre-launch to ascent, and another launch cannot start while the current mission runs.	Implemented MissionState as a Java enum for PRE_LAUNCH, ASCENT, ORBIT, DEPLOYMENT, SUCCESS and FAILURE; linked state changes to mission buttons.
4	**Automatic ascent and initial telemetry**	Launch and watch the rocket move upward. Confirm that displayed altitude and velocity change throughout ascent.	Introduced a simplified automatic flight model and connected its current values to the dedicated telemetry pane; manual piloting is not needed.
5	**Multithreaded simulation updates**	While the rocket is moving and telemetry is updating, interact with the interface to verify that the GUI remains responsive.	Moved the repeated flight calculations to a ScheduledExecutorService (50 ms updates). JavaFX elements are updated on the application thread via Platform.runLater.
6	**Target detection and satellite deployment**	Complete the ascent and wait for the 300 km apogee target. When deployment becomes available, click Deploy Satellite and watch it separate.	Added a simplified altitude/apogee milestone, state transition to ORBIT, and an animated satellite release ending in mission success; this is not full orbital mechanics.
7	**Mission completion and changing sky**	Deploy the satellite to display the success outcome. During another ascent, observe the background gradually change from sky to space.	Added mission success/failure states, safe stopping of normal progression and an altitude-dependent sky-to-space fade.
8	**Interactive mission emergency console**	Launch and wait for the engine-overheating event. Read the warning and select a response in the mission console.	Introduced a separate MissionEvent enum and response UI. Event prompts allow users to make decisions without pausing normal ascent.
9	**Emergency recovery and consequences**	At overheating, select emergency cooling or reduce throttle to recover. In another run, ignore the event or respond too late to trigger failure.	Implemented decision-dependent recovery, temporary throttle reduction, failure after unsafe decisions/timeouts, and recovery/failure visual notifications.

Rocket Mission Simulator - Final Product Backlog

#	Name	How to demo	Notes
10	**Modular simulation visuals**	Trigger a recovery or a failure and inspect the relevant visuals. In the source, see the separate FlightView, notification and overlay classes.	Refactored the increasingly complex SimulationPane into reusable visual components for flight graphics, engine effects, recovery feedback and outcomes.
11	**Animated mission log and event countdown**	Start a mission; watch terminal-style log entries appear with launch guidance. During an emergency, observe its countdown bar and choose an option.	Added queued/animated log messages, player instructions, interactive choice controls and a visible 20-second response countdown.
12	**GUI redesign and engine thrust animation**	Launch and watch the ignition sequence and engine exhaust. Check the refreshed mission controls, telemetry presentation and event interface.	Reworked the visual design and introduced animated engine ignition/plume graphics that respond to the engine state.
13	**Improved flight physics and automatic throttle**	Follow altitude, velocity, fuel and throttle through ascent. Observe throttle adjustments, propellant use, engine cutoff and the coast toward apogee.	Expanded the simplified model with gravity variation, atmospheric density, aerodynamic drag, propellant mass flow, acceleration limits, max-Q guidance and predicted apogee cutoff.
14	**Multiple altitude-triggered emergencies**	Continue a mission past overheating to encounter aerodynamic stress, propellant instability and attitude-control failure; choose appropriate responses.	Expanded the event system to four distinct events at different altitude thresholds, each with its own choices, recovery path and failure consequences.
15	**Final graphics, outcomes and restart**	Observe the updated satellite graphic and throttle-dependent plume. Complete or fail a mission, then choose Restart Mission or Exit Application.	Polished satellite and throttle animation; implemented reusable polymorphic SuccessOverlay/FailureOverlay classes and restart/exit buttons. Restart resets mission and simulation components.

Not implemented

- Manual throttle slider and player-controlled thrust/fuel use (implementation plan items 15-16). The flight model does adjust throttle and fuel automatically, but there is no slider in the GUI.
- Fuel exhaustion caused by player throttle choices as a distinct failure mechanic (item 17). Propellant consumption and failure to reach the target are simulated, but the player cannot control fuel usage directly.
- Left/right tilt buttons and player-controlled trajectory steering (items 18-19). The ascent is guided automatically.
- A user-steered orbital target point and failure from excessive tilt (items 20-21). The current target is reaching the simplified 300 km apogee milestone.
- Camera-following and zoom controls (item 22). The scenery changes with altitude, but there is no independent tracking/zoom camera.